

W05 Assignment: Explain Inheritance

What is inheritance and why is it important?

Inheritance is a principle of object-oriented programming that allows you to create a new class based on an existing one. The original class is called the base class or parent class, and the new one is called the child class or derived class. That is, a child class inherits its attributes and methods from the parent, and these are passed on to the child class without having to be rewritten.

For me, it's like a way to better organize code. For example, if several classes have similar characteristics, instead of repeating the same code in each one, it's better to put everything they have in a single class and then have the others inherit from it. It's like in real life: if we think of a family, the children inherit traits from their parents, but each one maintains its own characteristics.

Inheritance is important because it avoids code duplication. When we repeat code many times, the program becomes more difficult to maintain. On the other hand, if everything they have is in a base class, any change is made only once and affects all the classes that inherit from it. This also helps make the program easier to read and understand for oneself or other programmers, and this also makes the program easier to read and understand..

In the **Mindfulness** program I developed, I used inheritance to create a common structure for all the activities. First, I defined a base class called **Activity**, where I placed elements that all activities need, such as the name, description, duration, and some methods to run the activity.

The code snippet is as follows:

```
public abstract class Activity
{
    private string _name;
    private string _description;
    private int _durationSeconds;

    public void Run()
    {
        StartActivity();
        RunCore();
        EndActivity();
    }

    protected abstract void RunCore();
}
```

Then I created more specific classes that inherit from Activity, such as the BreathingActivity:

```
public class BreathingActivity : Activity
{
    public BreathingActivity()
        : base("Breathing Activity",
            "This activity will help you relax by guiding you through slow breathing.")
    {
    }
}
```

```
protected override void RunCore()
{
    Console.WriteLine("Breathe in... ");
    ShowCountdown(5);
}
}
```

In this example, you can clearly see the inheritance in the line:

```
public class BreathingActivity : Activity
```

This means that **BreathingActivity** inherits everything already defined in **Activity**. Thanks to this, I didn't have to rewrite methods like **Run()** or the activity's general logic. I only focused on what makes the breathing activity unique.

An important aspect of inheritance that I see is that it also allows for specialized behaviors. Although all activities share a common structure, each implements its own version of the **RunCore()** method. This makes the program flexible and easy to extend. For example, if in the future I wanted to add another activity, such as a **MeditationActivity**, I could create a new class that inherits from **Activity** and only write its specific behavior.

In conclusion, I think inheritance is a valuable resource that helps us reuse code, improves program organization, and makes maintenance easier. As a beginner, it has allowed me to understand how to create more structured programs and think about the design before I start writing code, instead of just focusing on making it work. This allows me to focus solely on what makes each activity different instead of rewriting common logic.

Made by: Ángel Cornejo
07/02/2026